

Rapport Projet - Vehicle Routing Problem with Time Windows (VRPTW)

Table des matières

1. Introduction
 2. Modélisation mathématique du problème
 3. Métaheuristiques implémentées
 4. Générateur de solutions et opérateurs de voisinage
 5. Protocole de tests et paramètres d'optimisation
 6. Résultats expérimentaux
 7. Analyse comparative et discussion
 8. Conclusions et perspectives
-

1. Introduction

Le Vehicle Routing Problem with Time Windows (VRPTW) est un problème d'optimisation combinatoire qui consiste à déterminer des itinéraires de véhicules minimisant la distance totale parcourue tout en satisfaisant les contraintes de capacité des véhicules et les fenêtres de temps de visite des clients.

Dans ce projet, deux métaheuristiques ont été implémentées pour résoudre le VRPTW, un algorithme génétique (GA), et la recherche taboue (TS). Ces deux méthodes vont dès lors être comparées pour évaluer leurs performances.

Notre objectif dans ce projet sera d'analyser les performances de ces deux approches en termes de qualité des solutions (distance minimale, nombre de véhicules), et d'essayer de trouver les hyperparamètres optimaux.

2. Modélisation mathématique du problème

2.1 Définition générale

Le VRPTW peut être formalisé comme un problème de programmation mathématique. Nous avons dès lors décrit :

Données du problème

- Un ensemble de clients $C = 1, 2, \dots, n$
- Un dépôt (le noeud 0 de notre ensemble de clients)
- Un ensemble de véhicules $V = 1, 2, \dots, K$.
- On se munit de la distance euclidienne entre deux nœuds i et j $d(i, j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$,
- La quantité demandé par le client i q_i
- La capacité Q que peut porter les véhicules de transport.

- La fenêtre de temps durant laquelle le client i peut être livré $[a_i, b_i]$ (ready_time, due_time)
- Et enfin le temps de livraison nécessaire pour le client i s_i

Variable de décisions et fonction objectif Pour définir une solution, nous avons ces variables de décisions :

- $x_{ijk} \in 0, 1$, qui est un booléen valant 1 si le véhicule k traverse l'arc (i, j) , 0 sinon.
- t_i l'instant d'arrivée au nœud i
- r_i l'instant de début de service au nœud i

Notre objectif sera dès lors de minimiser la distance total parcouru par les véhicules. $Z = \sum_i \sum_j \sum_k d(i, j) x_{ijk}$

Contraintes Nous sommes également soumis à des contraintes :

Contrainte 1 : Chaque client est visité exactement une fois $\sum_j \sum_k x_{ijk} = 1, \forall i \in C$

Contrainte 2 : Chaque véhicule doit revenir à son point de départ $\sum_i x_{ijk} = \sum_j x_{jik}, \forall i, j \in V \cup C, \forall k \in V$

Contrainte 3 : La charge totale d'une route ne peut pas dépasser la capacité maximale d'un véhicule $\sum_{i \in route_k} q_i \leq Q, \forall k \in V$

Contrainte 4 : Un véhicule doit arriver dans la fenêtre de temps d'un client $a_i \leq t_i \leq b_i, \forall i \in C \cup 0$

Contrainte 5 : Un véhicule doit pouvoir arriver dans la fenêtre de temps du voisin suivant $t_j \geq t_i + s_i + d(i, j)$ si $x_{ij} = 1$

2.2 Structures de données utilisées

Pour fonctionner, nous avons implémenté plusieurs structures essentiels pour la recherche d'une solution optimisé :

- **Location**, représentant les coordonnées cartésiennes (x, y) d'un nœud.
- **TimeWindow**, qui est l'intervalle $[a_i, b_i]$ pour la contrainte de temps.
- **Client** représente un client avec une **Location**, la quantité demandé q_i , une **TimeWindow**, et le temps nécessaire à la livraison s_i .
- **Depot** quant à lui représente le nœud de départ/arrivée. Il possède une **Location** et définit la capacité Q des véhicules.
- **Route** est une séquence ordonnée de **Client** desservis par un véhicule unique.
- **Solution** est l'ensemble de **Route** couvrant tous les **Client**.

- `VRPTProblem`, enfin, représente l’instance complète pour définir le problème.

2.3 Fonction d’évaluation

Pour une solution donnée, trois indicateurs vont être pris en compte pour comparer les deux algorithmes : la distance totale, qui est la somme des distances de toutes les routes, et le nombre de véhicules utilisé pour la solution.

3. Métaheuristiques implémentées

3.1 Algorithme Génétique (GA)

Principes fondamentaux L’algorithme génétique s’inspire de la théorie de l’évolution. À partir d’une population initiale de solutions, il crée itérativement une nouvelle génération en effectuant ces étapes :

- On sélectionne les meilleurs individus de la génération précédentes.
- On effectue les croisements (reproduction avec recombinaison)
- Puis les mutations
- Enfin, on remplace la nouvelle générations a partir des nouvelles solutions (individus), en gardant les meilleurs solutions.

La population initiale de taille `POPULATION_SIZE` sera générée à partir de la solution initiale, elle-même générée en utilisant le générateur de solutions décrit dans la section 4.

Opérateurs génétiques Pour le bon fonctionnement de l’algorithme, nous avons défini des opérateurs génétiques pour concevoir la génération suivante, à partir d’une génération initiale. Pour modifier facilement le comportement de l’algorithme (plus conservatif, plus long mais plus précis, etc.) Nous avons défini des hyperparamètres, facilement modifiable depuis le fichier `src/hyperparameters.py`. Voici les hyperparamètres utilisé pour notre GA, que nous décrirons plus en détails dans les sections suivante :

```
POPULATION_SIZE
GENERATIONS
CROSSOVER_RATE
MUTATION_RATE
ELITE_SIZE
TOURNAMENT_SIZE
```

Sélection Pour chaque itération, deux parents vont être sélectionnés par “tournoi”. Pour ce faire :

- On sélectionne aléatoirement `t` individus de la population (`TOURNAMENT_SIZE = t`)

- On choisi ensuite la meilleure solution parmi ces t individus.
- Ce processus est répété pour obtenir 2 parents

Cela permet une pression sélective un peu plus équilibrée par rapport à la sélection proportionnelle.

Croisement Le croisement a pour objectif de générer un enfant à partir de deux parents :

- On choisit deux points de coupure aléatoires : $p1$ et $p2$
- Ensuite, on copie le segment $[p1, p2]$ du parent 1 vers l'enfant
- Enfin, on remplit les positions restantes en utilisant l'ordre du parent 2, en respectant les clients déjà présents et en effectuant les mappages cycliques nécessaires.

Pour limiter la diversité (qui mènerais à des générations ne conservant pas assez les bonnes solutions), on applique une probabilité d'application CROSSOVER_RATE.

Mutation Des mutations peuvent être appliqué sur chaque point. Trois types de mutations sont appliquées aléatoirement :

- Inversion de segment (Reverse) : inverser un segment d'une route
- Insertion : retirer un client aléatoire d'une route et l'insérer ailleurs
- Échange inter-routes : échanger deux clients de routes différentes

Pour la même raison que le croisement, on applique une probabilité d'application MUTATION_RATE, pour limiter les applications de mutation.

Élitisme Pour chaque génération, on sélectionne les $t = \text{ELITE_SIZE}$ meilleures solutions de la génération actuelle. Elles sont alors automatiquement copiées dans la nouvelle génération sans modifications, garantissant que la meilleure solution n'est jamais perdue.

Termination Enfin, l'algorithme s'exécutera pour un nombre fixe de générations défini par l'hyperparamètre GENERATIONS.

3.2 Recherche Taboue (TS)

Principes fondamentaux La Recherche Taboue est un algorithme de recherche locale avec mémoire taboue qui échappe aux optima locaux en interdisant temporairement certains mouvements.

La solution initiale sera générée en utilisant le générateur de solutions décrit dans la section 4.

Opérateurs de voisinage De la même façon que notre algorithme génétique, notre recherche Taboue utilise des hyperparamètres, défini également dans `src/hyperparameters.py`, pour faciliter la modification du comportement de l'algorithme.

MAX_ITERATIONS
TABU_TENURE
NEIGHBORHOOD_SIZE
ASPIRATION_CRITERIA

Le voisinage est généré en utilisant plusieurs mouvements de base :

- 2-opt, pour inverser un segment d'une route.
- Réassignation, pour déplacer un client d'une route à une autre.
- Échange, pour modifier deux clients entre deux routes différentes.

À chaque itération, un ensemble de NEIGHBORHOOD_SIZE mouvements candidats est généré aléatoirement à partir de tous les mouvements possibles.

Gestion de la mémoire taboue Pour le bon fonctionnement de l'algorithme Taboue, nous utilisons plusieurs variables :

- Une liste taboue, qui contient les derniers mouvements interdits avec la durée qui définit depuis quand le mouvement est interdit.
- Un Tabu tenure, qui est la durée pendant laquelle un mouvement reste interdit (TABU_TENURE). Un mouvement est marqué comme taboue s'il figure dans la liste taboue.
- Et un critère d'aspiration, qui définit si un mouvement taboué peut être accepté s'il conduit à une solution meilleure que la meilleure solution trouvée jusqu'à présent (ASPIRATION_CRITERIA)

Sélection du meilleur voisin Pour sélectionner le meilleur voisin, on filtre dans un premier temps les solutions faisables (respect des contraintes), puis on évalue chaque voisin non-taboué ou satisfaisant le critère d'aspiration (si activé), et enfin on choisit le voisin avec la meilleure fonction objectif.

La fonction objectif pour la comparaison est : $f(\text{solution}) = (\text{num_vehicles}, \text{total_distance})$

Termination L'algorithme s'exécute pour MAX_ITERATIONS itérations.

4. Générateur de solutions et opérateurs de voisinage

4.1 Générateurs de solutions initiales

Pour que nos algorithmes fonctionnent, nous utilisons un générateur de solutions pour que nos métaheuristiques puissent avoir un point de départ. Pour ce faire,

notre générateur va s'appuyer sur deux algorithmes.

Nearest-Neighbor C'est un algorithme glouton relativement simple :

- On Initialise une solution vide avec des routes vierges
- Tant qu'il existe des clients non assignés :
 - On crée une nouvelle route
 - Tant que la route n'est pas pleine et qu'il reste des clients :
 - * On trouve le client non assigné le plus proche de la dernière position, puis on l'ajoute s'il respecte les contraintes (capacité et fenêtres de temps). Le cas échéant, on passe à la route suivante.

Random Solution Si le Nearest-Neighbor ne fonctionne pas, on passe sur une heuristique aléatoire. Dès lors, on permute aléatoirement l'ordre des clients, et on assigne ensuite chaque client séquentiellement à la première route l'acceptant (capacité + fenêtres respectées)

4.2 Opérateurs de voisinage

Pour définir quelle solution est voisine d'une autre, il faut des opérateurs pour les relier. Nous avons défini trois opérateurs de voisinages qui nous semblaient pertinent.

2-opt Le 2-opt est caractérisé par un déplacement d'un segment de route à un autre endroit de cette même route :

... - i - $i+1$ - ... - j - $j+1$ - ... devient ... - i - j - ... - $i+1$ - $j+1$ - ...

Mathématiquement, le gain de distance est : $\delta = d(i, j+1) + d(j, i+1) - d(i, i+1) - d(j, j+1)$

Insertion L'insertion est un déplacement d'un client d'une route à une autre. On sélectionne un client c d'une route source et une position dans une route destination, et ensuite on retire c de sa route actuelle pour l'insérer à la nouvelle position dans la route destination.

Échange Enfin, l'échange est, comme son nom l'indique, un échange de deux clients entre routes différentes. Pour ce faire, on sélectionne deux clients $c1$ (route $r1$) et $c2$ (route $r2$) et on permute leurs positions dans leurs routes respectives

5. Protocole de tests et paramètres d’optimisation

5.1 Jeux de données

Pour effectuer nos tests, nous disposons de dix instances de test couvrant différentes tailles et caractéristiques :

Dataset	Capacité d’un véhicule	temps d’ouverture du dépôts
data101	200	230
data102	200	230
data1101	200	240
data1102	200	240
data111	200	230
data112	200	230
data1201	1000	960
data1202	1000	960
data201	1000	1000
data202	1000	1000

Les différences entre les datasets non visible dans ce tableau résident dans les différences entre les fenêtres de temps et les positions de chacun des clients.

5.2 Evolution du protocole de tests

De nombreuses séries de tests ont été exécutées pour affiner le modèle, jusqu’à arriver à notre protocole actuel, qui consiste à :

- Sélectionner les hyperparamètres (via `src/hyperparameters.py`)
- On lance un tests sur chaque metaheuristique, pour chaque dataset, dans la version VRP (sans fenêtre de temps) et VRPTW (avec fenêtre de temps).
- Pour chaque dataset, on génère des plots des solutions trouvé par nos algorithmes et des performances de chacun, et on fait un tableau comparatifs de nos solutions.

Dans la suite de ce rapport, nous allons comparer 4 séries de tests effectués, en variant nos hyperparamètres. N’ayant pas eu le temps de faire des séries de tests conséquents, certains hyperparamètres n’ont pas été réellement tester, par conséquents les tests présentés ici seront systématiquement avec `TS_DEFAULT_ASPIRATION_CRITERIA = True`.

TEST 1 L’objectif était d’établir un premier tests avec des hyperparamètres très modérés, pour une vitesse d’exécution très rapide.

```
GA_DEFAULT_POPULATION_SIZE = 50
GA_DEFAULT_GENERATIONS = 100
GA_DEFAULT_CROSSOVER_RATE = 0.8
GA_DEFAULT_MUTATION_RATE = 0.2
```

```

GA_DEFAULT_ELITE_SIZE = 2
GA_DEFAULT_TOURNAMENT_SIZE = 3
TS_DEFAULT_MAX_ITERATIONS = 1000
TS_DEFAULT_TABU_TENURE = None
TS_DEFAULT_NEIGHBORHOOD_SIZE = 100

```

Nous obtenons dès lors ces résultats :

VRP (sans fenêtres de temps)

Problem	GA Distance	GA Vehicles	TS Distance	TS Vehicles
data101	1090.93	8	1005.67	8
data102	1100.40	8	1003.37	8
data1101	1231.42	9	1239.33	9
data1102	1224.81	9	1234.05	9
data111	1097.97	8	955.41	8
data112	1104.71	8	928.88	8
data1201	758.30	2	827.50	2
data1202	776.91	2	832.01	2
data201	800.91	2	802.73	2
data202	801.18	2	802.73	2

VRPTW (avec fenêtres de temps)

Problem	GA Distance	GA Vehicles	TS Distance	TS Vehicles
data101	2478.84	33	1886.31	23
data102	2279.57	27	1704.22	22
data1101	2459.94	25	1933.59	20
data1102	2479.21	22	1867.03	21
data111	1579.93	17	1317.30	17
data112	1232.12	12	1193.64	13
data1201	2411.84	12	1838.08	13
data1202	2050.96	11	1554.82	11
data201	2194.04	12	1465.96	13
data202	1791.14	12	1395.96	13

On remarque dans un premier temps la différence assez importante des résultats sur l'algorithme génétique, qui utilise bien plus de véhicules et est bien moins performant de manière générale sur les problèmes plus complexes avec les fenêtres de temps. Lors des tests, on a pu remarquer une convergence bien trop rapide (à la génération 50).



Figure 1: vrp_comparison.png

TEST 2 Pour notre test 2, notre objectif était de casser la convergence prématurée, et plus généralement améliorer les performances par une augmentation importantes des ressources.

```
GA_DEFAULT_POPULATION_SIZE = 400
GA_DEFAULT_GENERATIONS = 500
GA_DEFAULT_CROSSOVER_RATE = 0.90
GA_DEFAULT_MUTATION_RATE = 0.08
GA_DEFAULT_ELITE_SIZE = 1
GA_DEFAULT_TOURNAMENT_SIZE = 2
TS_DEFAULT_MAX_ITERATIONS = 2000
TS_DEFAULT_TABU_TENURE = 10
TS_DEFAULT_NEIGHBORHOOD_SIZE = 150
```

Le passage à population = 400 et tournament = 2 aide à casser la convergence prématurée, tandis que mutation = 0.08 réinjecte bien plus de nouveauté sans rendre l'évolution chaotique. nous avons également augmenté le nombre de génération à 500. Le tournament size à également été réduit pour augmenter la flexibilité.

Côté TS, nous avons bien augmenté le nombre d'itérations maximal à 2000 pour tenter de l'affiner, et neighborhood = 150 te donne plus d'options locales. Nous avons également ajouté un Tabu tenure à 10 pour conserver la liste taboué sur

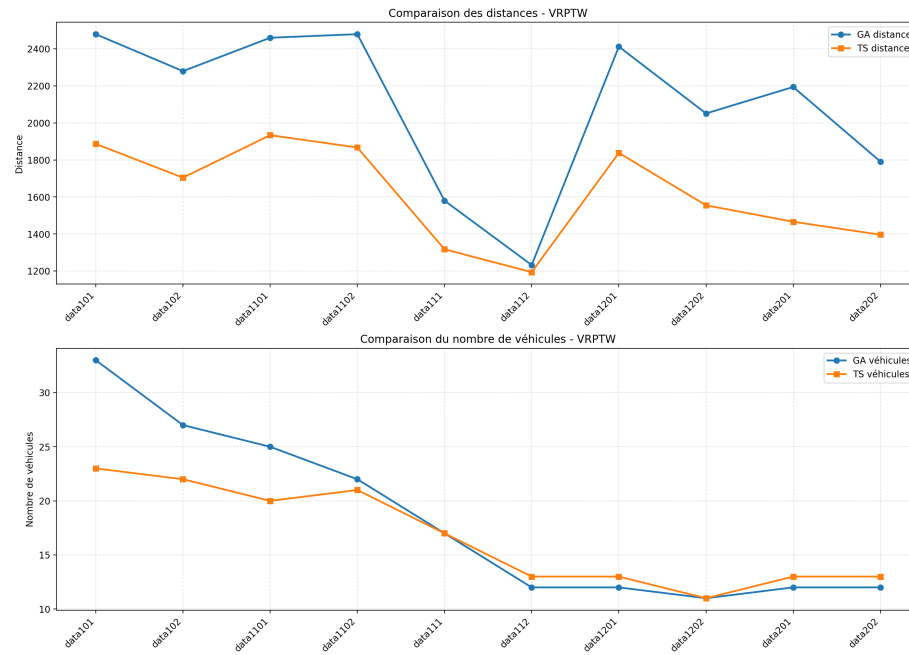


Figure 2: vrptw_comparison.png

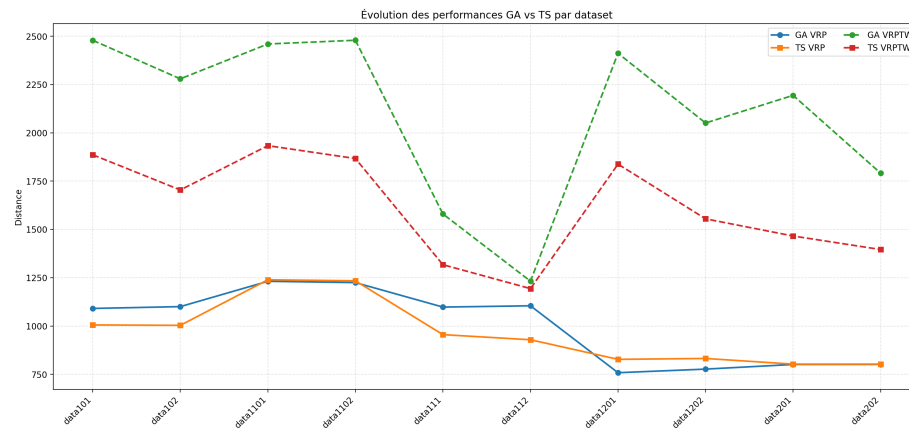


Figure 3: ga_ts_evolution_by_dataset.png

au moins 10 itérations.

VRP

Problem	GA Distance	GA Vehicles	TS Distance	TS Vehicles
data101	1072.26	8	936.11	8
data102	1064.82	8	943.98	8
data1101	1191.41	9	1185.48	9
data1102	1185.23	9	1252.44	9
data111	1067.70	8	937.50	8
data112	1064.89	8	910.91	8
data1201	744.49	2	792.95	2
data1202	745.48	2	762.43	2
data201	764.20	2	795.73	2
data202	764.80	2	800.66	2

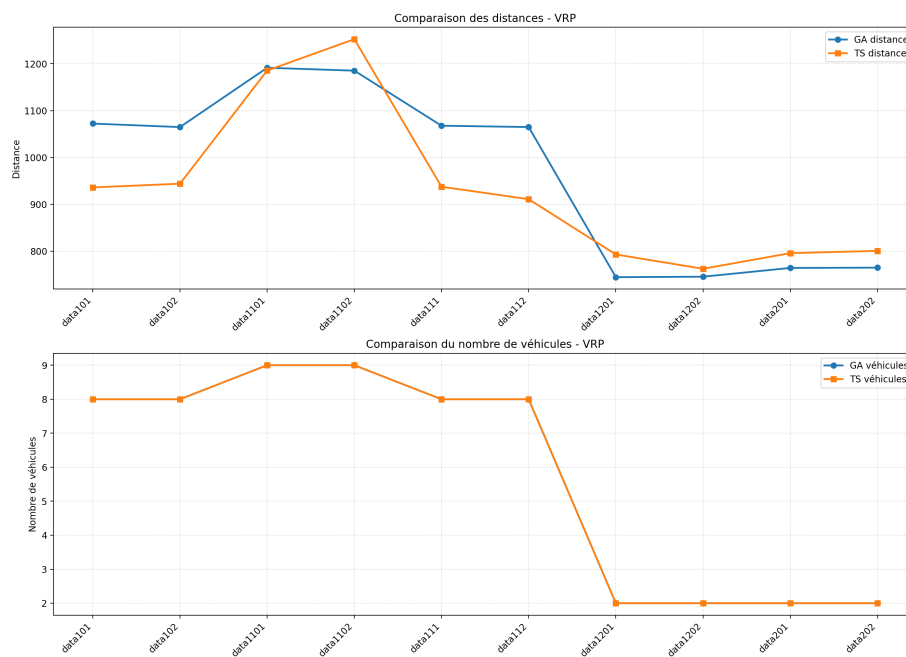


Figure 4: vrp_comparison.png

VRPTW

Problem	GA Distance	GA Vehicles	TS Distance	TS Vehicles
data101	2043.60	26	1792.60	24
data102	1958.14	22	1669.74	22
data1101	2298.05	23	1877.52	21
data1102	2216.75	20	1817.41	20
data111	1511.41	18	1342.16	17
data112	1165.72	12	1118.34	13
data1201	2242.06	12	1795.48	13
data1202	1948.61	10	1638.55	12
data201	1977.78	12	1516.68	14
data202	1605.25	11	1266.23	12

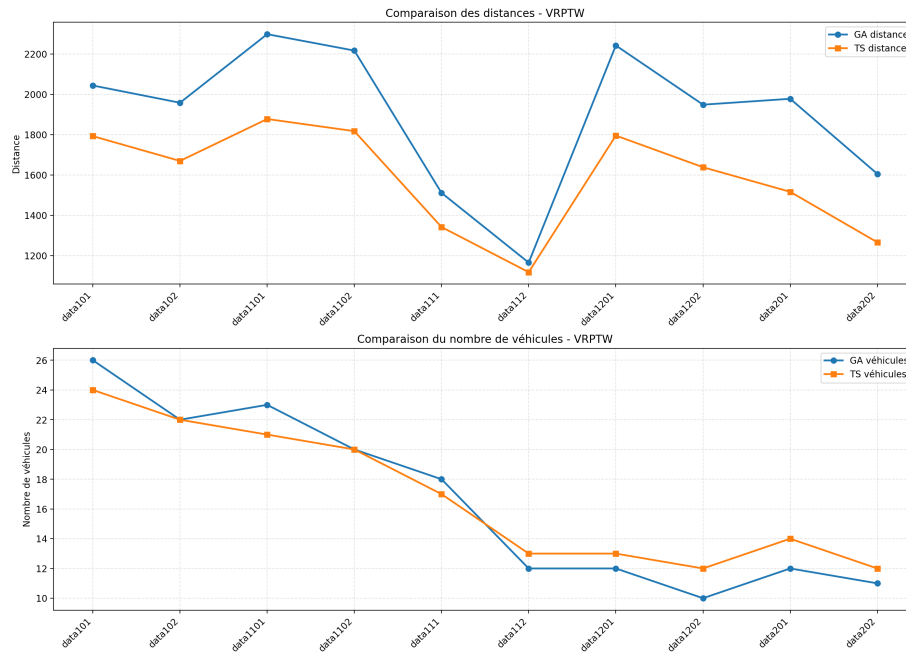


Figure 5: vrptw_comparison.png

Avec ces modifications, les deux métaheuristique ont gagné en efficacité, notamment le GA qui à bien rattrapé son retard, même s'il y a encore des écarts importants sur le VRPTW (près de 400 de distance supplémentaires par rapport au TS, même si le GA trouve des solutions avec un nombre de véhicule réduit. $GA_DEFAULT_POPULATION_SIZE = 400$

TEST 3 : Réduction des générations GA Lors de nos précédents tests, nous avons remarqué que la convergence avait lieu aux alentours de 250/300.

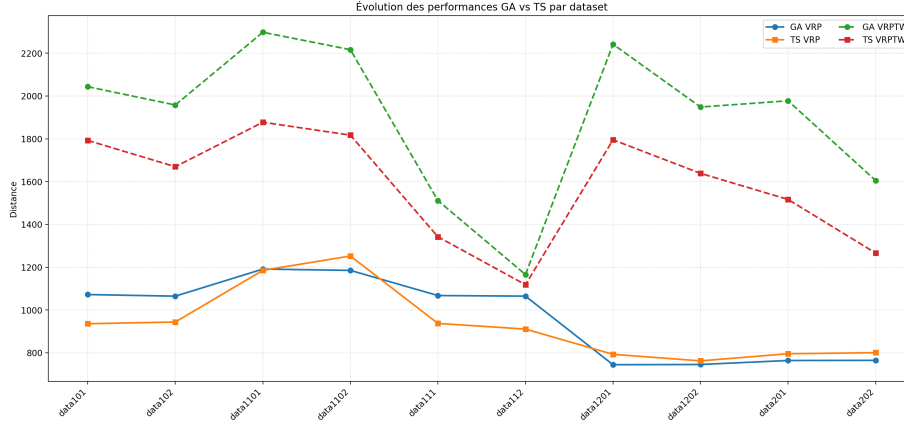


Figure 6: ga_ts_evolution_by_dataset.png

Etant donné le temps conséquents pour effectuer les calculs (~1h30 sur nos machines personnelles), en particulier la GA, nous avons réduit le nombre de génération pour le GA, en passant de 500 à 300 générations, l'objectif étant d'accélérer les calculs pour ne pas être trop pénalisé par le temps de calculs, sans perdre en qualité.

```
GA_DEFAULT_POPULATION_SIZE = 400
GA_DEFAULT_GENERATIONS = 300
GA_DEFAULT_CROSSOVER_RATE = 0.90
GA_DEFAULT_MUTATION_RATE = 0.08
GA_DEFAULT_ELITE_SIZE = 1
GA_DEFAULT_TOURNAMENT_SIZE = 2
TS_DEFAULT_MAX_ITERATIONS = 2000
TS_DEFAULT_TABU_TENURE = 10
TS_DEFAULT_NEIGHBORHOOD_SIZE = 150
```

VRP

Problem	GA Distance	GA Vehicles	TS Distance	TS Vehicles
data101	1070.07	8	950.10	8
data102	1074.11	8	910.88	8
data1101	1185.18	9	1247.79	9
data1102	1174.83	9	1202.55	9
data111	1031.84	8	974.05	8
data112	1049.58	8	935.98	8
data1201	753.89	2	781.99	2
data1202	744.26	2	768.24	2
data201	768.07	2	798.72	2

Problem	GA Distance	GA Vehicles	TS Distance	TS Vehicles
data202	783.04	2	795.35	2

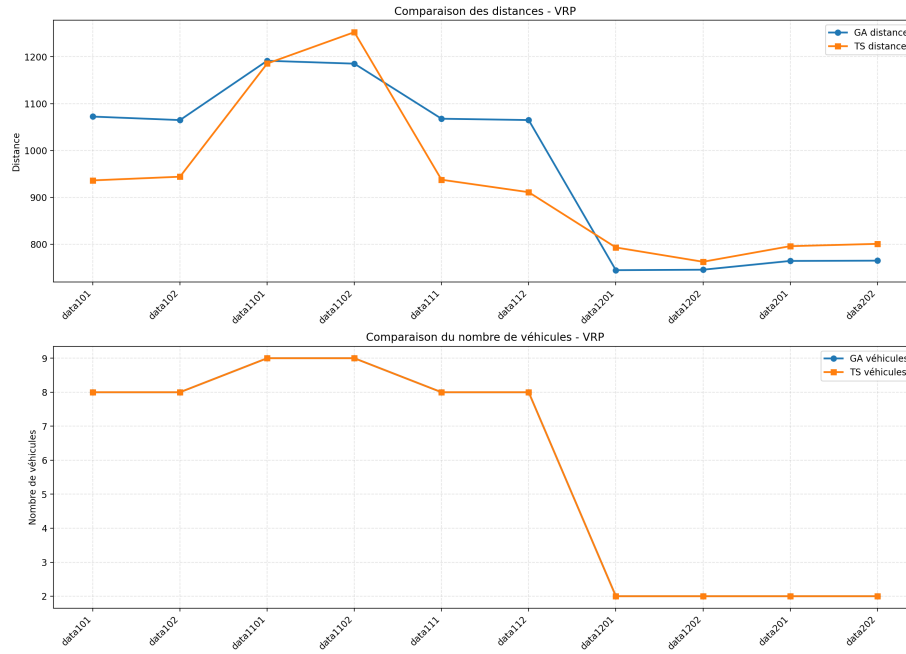


Figure 7: vrp_comparison.png

VRPTW

Problem	GA Distance	GA Vehicles	TS Distance	TS Vehicles
data101	2197.03	28	1813.39	23
data102	2032.28	23	1619.30	21
data1101	2158.56	22	1996.94	21
data1102	2275.61	21	1808.94	21
data111	1533.89	18	1391.62	18
data112	1153.80	12	1154.97	13
data1201	2319.18	12	1888.66	12
data1202	1982.43	10	1505.24	11
data201	2020.47	12	1447.47	14
data202	1661.91	11	1256.18	12

Nous pouvons dès lors remarquer des résultats similaires à notre test 2, mais avec un temps d'exécution amoindries (réduit de ~30% !).

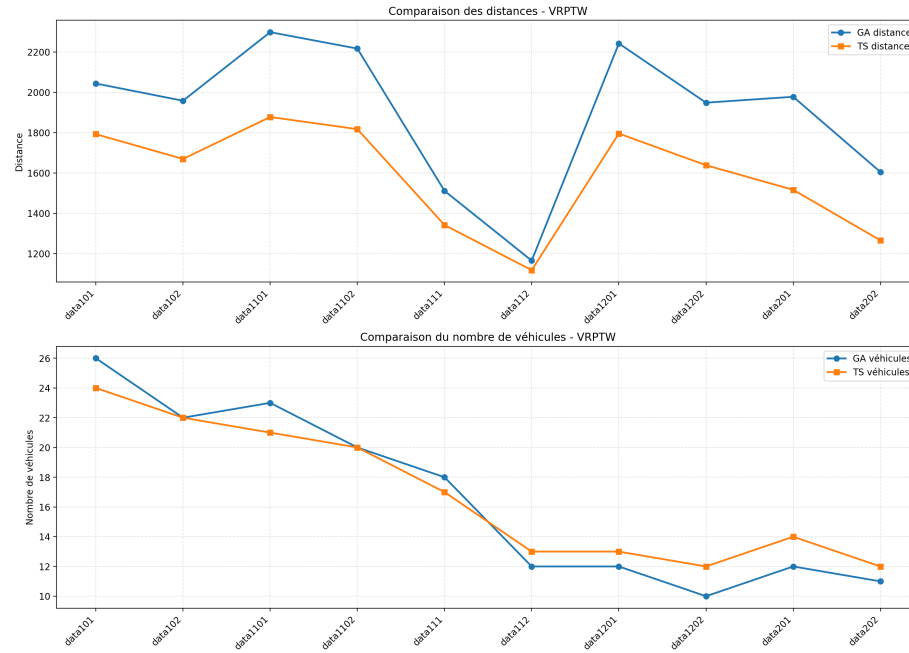


Figure 8: vrptw_comparison.png

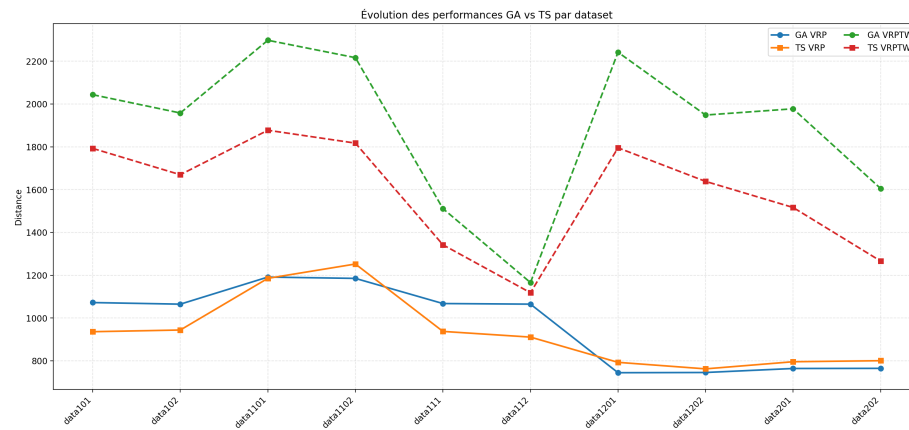


Figure 9: ga_ts_evolution_by_dataset.png

TEST 4 : Configuration finale optimisée Après de multiple modifications légère, nous avons fini sur ces hyperparamètres :

```
GA_DEFAULT_POPULATION_SIZE = 400
GA_DEFAULT_GENERATIONS = 500
GA_DEFAULT_CROSSOVER_RATE = 0.90
GA_DEFAULT_MUTATION_RATE = 0.12
GA_DEFAULT_ELITE_SIZE = 1
GA_DEFAULT_TOURNAMENT_SIZE = 2
TS_DEFAULT_MAX_ITERATIONS = 2000
TS_DEFAULT_TABU_TENURE = 12
TS_DEFAULT_NEIGHBORHOOD_SIZE = 150
TS_DEFAULT_ASPIRATION_CRITERIA = True
```

Pour le GA, nous avons augmenté le taux de mutation à 0.12 (et finalement remis le nombre de générations à 500 pour être en cohérence avec ce taux), et pour le TS nous avons simplement augmenté la tabu tenure à 12 (le max_iterations ne semblant pas influencer plus que ça sur le résultat). Nous avons dès lors obtenu ces résultats :

VRP

Problem	GA Distance	GA Vehicles	TS Distance	TS Vehicles
data101	1039.16	8	925.01	8
data102	1060.29	8	922.76	8
data1101	1175.37	9	1184.65	9
data1102	1177.60	9	1228.14	9
data111	1036.28	8	930.87	8
data112	1048.44	8	955.53	8
data1201	743.40	2	776.51	2
data1202	744.97	2	762.28	2
data201	766.75	2	794.07	2
data202	757.02	2	787.61	2

VRPTW

Problem	GA Distance	GA Vehicles	TS Distance	TS Vehicles
data101	2031.37	25	1823.19	23
data102	1852.66	23	1620.08	22
data1101	2169.20	22	1972.29	21
data1102	2217.96	20	1749.69	19
data111	1442.25	16	1310.47	16
data112	1152.38	12	1131.91	13
data1201	2223.52	12	1959.07	13

Problem	GA Distance	GA Vehicles	TS Distance	TS Vehicles
data1202	1970.39	9	1529.92	12
data201	1910.61	11	1401.06	13
data202	1579.10	12	1299.62	12

Nous obtenons dès lors ce graphe :

7. Analyse comparative et discussion

7.1 Performance en mode VRP (sans fenêtres de temps)

Tendances générales De manière générale, le TS domine fortement pour les problèmes avec les temps d'ouverture du dépôts restreints (data101, data102, data111, data112), TS gagne constamment sur ces instances.

L'algorithme génétique quant à lui s'est montré peu efficace dans les premiers essais, et nécessite des temps de calculs bien plus importants pour être efficace. Cependant, celui-ci, si bien configuré, semble pouvoir s'améliorer continuellement, à condition de lui donner un nombre de génération conséquent (et donc du temps et un pc puissant, ce que nous n'avions pas)

Analyse statistique

Métrique	TS Performance	GA Performance
Stabilité	Très stable	Améliore progressivement
Temps d'exécution	Rapide/Moyen	Moyen/Très long

La transition dans les performances s'explique par la structure des problèmes : - Petits problèmes : TS converge rapidement vers optima local de bonne qualité - Moyens/grands : GA explore mieux l'espace de solutions malgré le temps limité

7.2 Performance en mode VRPTW (avec fenêtres de temps)

Les fenêtres de temps rendent le problème significativement plus difficile. Le nombre de véhicules augmente souvent de 3-4x comparé au VRP.

Impact des fenêtres de temps Distance supplémentaire due aux fenêtres :

- data101 : 1005.67 (VRP) → 1823.19 (VRPTW) = +81%
- data201 : 794.07 (VRP) → 1401.06 (VRPTW) = +76%

Nombre de véhicules nécessaires :

- VRP : moyenne ~5 véhicules

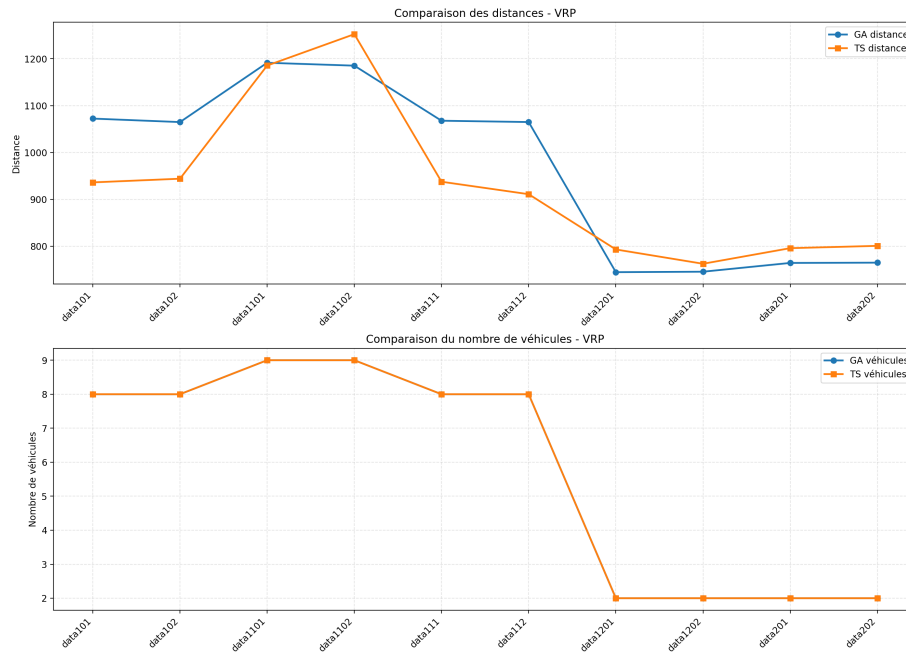


Figure 10: vrp_comparison.png

- VRPTW : moyenne ~19 véhicules

Performance comparative Tabu Search reste dominant en VRPTW moyenne-petite (TEST 4) :

- Meilleur sur data101, data102, data1101, data1102, data111
- L'intensification de la recherche locale aide à respecter les fenêtres

L'algorithme génétique semble cependant plus compétitif la encore sur des instances complexes, (pour peu qu'on lui donne le nombre de génération nécessaire...)

7.3 Impact des hyperparamètres

Population et Générations du GA Les hyperparamètres sont absolument essentiels pour obtenir des résultats performants. En effet, nous avons pu remarquer à travers nos tests à quel point la différence des résultats peuvent réellement varier, en particulier entre le test 1 et le test 4, qui a permis une évolution significatif en réglant correctement les paramètres pour gérer correctement l'exploration et la convergence, et même faire perdre du temps pour peu de valeurs ajoutée, comme par exemple avec le test 2 et 3, qui ont des résultats similaires alors que le test 3 à une réduction de générations de 200, mais avec

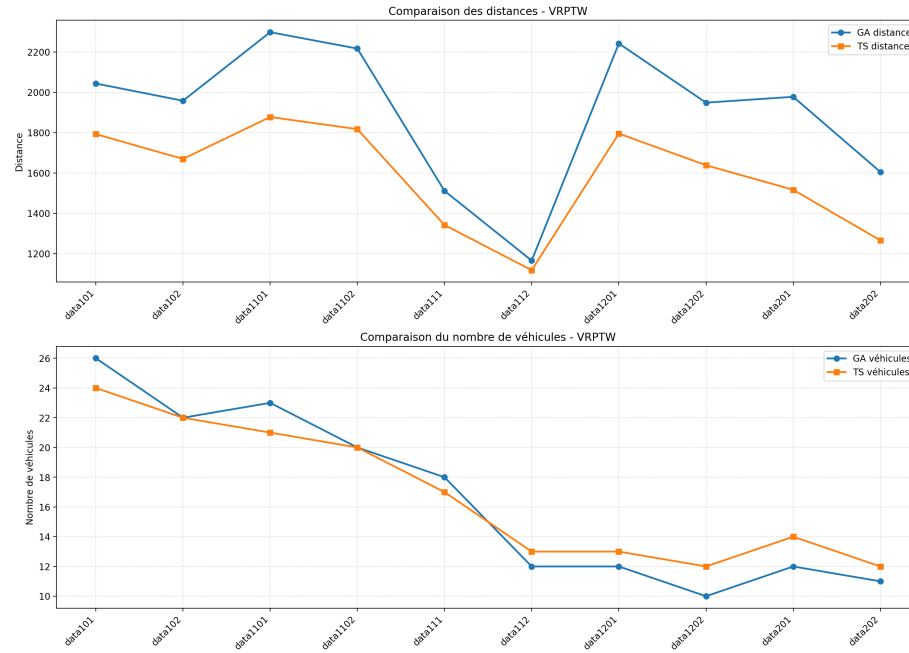


Figure 11: vrptw_comparison.png

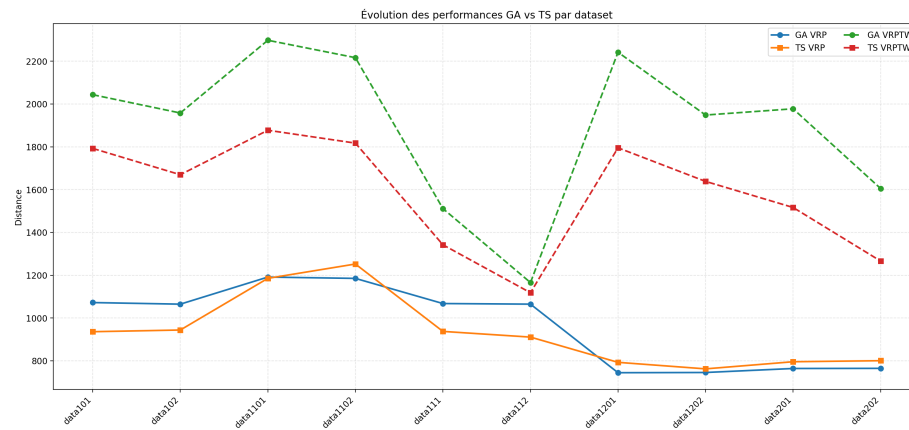


Figure 12: ga_ts_evolution_by_dataset.png

des telos réduit de ~30%. Ainsi, la configuration du test 3 offre bon compromis temps/qualité.

7.4 Faisabilité

Pendant notre développement, de nombreux problèmes de faisabilité ont mené à un retard dans le projet (et donc moins de temps pour effectuer les tests. . .) Cependant les algorithmes présentent donc des résultats qui respectent bien les contraintes.

8. Conclusions et perspectives

8.1 Synthèse des résultats

En résumé, La recherche Taboue à été supérieure dans la plupart des tests, cependant nous avons pu remarquer que plus les problèmes se complexifiaient et les puissances de calculs des algorithmes augmentaient, plus l'écart des résultats entre les deux algorithmes se resserraient. L'algorithme génétique, pour peu que la puissance calculatoire de l'ordinateur soit suffisante, semble plus adapté pour être performant sur des configurations plus complexe

8.2 Améliorations futures possibles

Pour le futur, il serait pertinent de poursuivre les tests avec des pc plus puissants, des algorithmes optimisés, et peut être remplacer des opérateurs de voisinages plus pertinent. Peut être du 3-opt, Lin-Kernighan pour TS; croisement adaptatif pour GA ?) Nous pourrions aussi essayer de repartir d'une solution TS, qui est déjà très efficace, puis s'en servir comme solution initiale pour un GA.

8.4 Limitations du projet

Du fait de la performance limitée de nos ordinateurs, il est difficile d'obtenir des résultats très pertinent sans passer plusieurs jours à effectuer les calculs. Nos tests ont été effectués pour viser un temps d'exécution des séries de tests n'excédant pas 2h, mais en contrepartie les résultats (notamment sur le GA) sont probablement sous évalués.

8.5 Conclusion finale

Ce projet démontre que deux métaheuristiques différentes peuvent produire des résultats complémentaires sur le problème VRPTW. La Recherche Taboue excelle en optimisation rapide, tandis que l'Algorithme Génétique propose une meilleure exploration globale. Le choix entre les deux dépend fortement de la nature de l'instance (taille, structure spatiale, contraintes) et des objectifs du décideur (temps vs qualité).

Les configurations finales validées (GA : 400 pop, 500 gen, 0.12 mutation ; TS : 2000 iter, tenure 12, neighborhood 150) constituent une base solide pour résoudre des instances réelles du VRPTW.

Annexe : Guide d'exécution

Dépendances

```
numpy==1.24.3
matplotlib==3.7.1
pandas==2.1.0
scipy==1.10.1
folium==0.14.0
```

Installation

depuis la racine du projet :

```
python -m venv
source venv/bin/activate
pip install -r requirements.txt
```

Modification des hyperparamètres :

il suffit d'aller modifier les valeurs présents dans `src/hyperparameters.py`

Exécution des tests

Dans la racine du projet, lancer :

```
python run_experiments.py
```

Les résultats sont sauvegardés dans `results/test{i}/comprehensive_results.json` et les graphiques dans `results/test{i}/plots/`. Pour chaque datasets, on peut également visualiser le résultats des deux métaheuristiques trouvé en mode VRP/VRPTW dans `results/test{i}/solutions/data{XXX}`

Structure des fichiers

<code>run_experiments.py</code>	# fichier python pour tester
<code>data/</code>	# Les datasets
<code>results/</code>	# emplacements des résultats des tests
<code>src/</code>	
<code>models.py</code>	# Structures de données (Solution, Route, Client, etc.)
<code>genetic_algorithm.py</code>	# Implémentation GA
<code>tabu_search.py</code>	# Implémentation TS
<code>hyperparameters.py</code>	# Paramètres globaux

neighborhood.py	# Opérateurs de voisinage
solution_generator.py	# Générateurs de solutions initiales
distance_utils.py	# Calcul distances et évaluation
data_loader.py	# Chargement des instances .vrp